Solr vs. Elasticsearch: Who's The Leading Open Source Search Engine?

Asaf Yigal

Sep 27th, 2016 | 4 comments | Comparison, Elasticsearch

More than ever, this is the time of cloud and data growth. Today's applications generate data in petabytes and zettabytes while everyone still demands faster and faster performance. However, as the data piles up, searching through all of that information effectively quickly becomes a substantial back-end challenge.

In this post, I will compare two of the most popular open source search engines, Solr and ElasticSearch. Both were built on top of the Apache Lucene open source platform, so several of their functionalities are very similar. However, there are great differences in terms of ease of deployment, scalability, and other functionalities as well.

More on the subject:

- Heka vs. Logstash: A Tale of Two Log Processors
- Fluentd vs. Logstash: A Comparison of Log Collectors
- Chef vs. Puppet: Methodologies, Concepts, and Support

About Apache Solr

Apache Solr is an open source search platform built on a Java library called Lucene. It offers Apache Lucene's search capabilities in a user-friendly way. Having been an industry player for almost a decade, it is a mature product with a strong and broad user community. It offers distributed indexing, replication, load-balanced querying, and automated failover and recovery. If it is deployed correctly and then managed well, it's capable of becoming a highly reliable, scalable, and fault-tolerant search engine. Quite a few internet giants such as Netflix, eBay, Instagram, and Amazon (CloudSearch) use Solr because of its ability to index and search multiple sites.

The major feature list includes:

- Full-text search
- Highlighting

- Faceted search
- Real-time indexing
- Dynamic clustering
- Database integration
- NoSQL features and rich document handling (Word and PDF files, for example)

About Elasticsearch

Elasticsearch is an open source (Apache 2 license), distributed, a RESTful search engine built on top of the Apache Lucene library.

Elasticsearch was introduced a few years after Solr. It offers a distributed, multitenant-capable, full-text search engine with an HTTP web interface (REST) and schema-free JSON documents. The official client libraries for Elasticsearch are available in Java, Groovy, PHP, Ruby, Perl, Python, .NET, and Javascript.

The distributed search engine includes indices that can be divided into shards, and each shard can have multiple replicas. Each Elasticsearch node can have one or more shards, and its engine also acts as a coordinator to delegate operations to the correct shard(s).

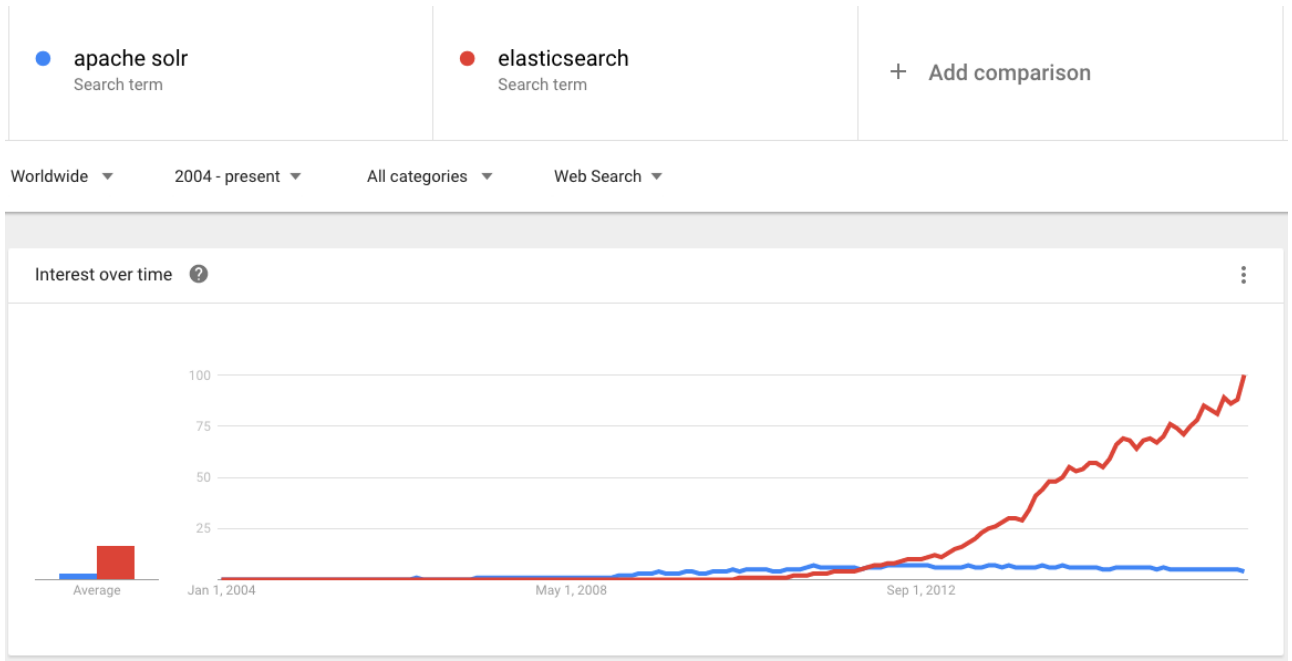
Elasticsearch is scalable with near real-time search. One of its key features is multi-tenancy.

The major feature list includes:

- Distributed search
- Multi-tenancy
- An analyzer chain
- Analytical search
- Grouping & aggregation

For more about Elasticsearch, you can see our Elasticsearch tutorial as well as our archive of posts about it and our guide to the ELK Stack as a whole.

The Trend



Before we begin, let's check Google Trends for both products. Google Trends shows that Elasticsearch has a great traction in comparison to Solr, but that does not mean that Apache Solr is dead. Although some might think otherwise, Solr is still one of the most popular search engines with a robust community and open source support.

Installation & Configuration

Elasticsearch is easy to install and very lightweight compared to Solr. The current version (6.2.0) of Solr's distribution package size is around 150 MB while the current version (2.4.0) of Elasticsearch distribution package size is only 26.1 MB. In addition, you can install and run Elasticsearch within a few minutes.

More on the subject:

- Musings in YAML – Tips for Configuring Your Beats
- Fluentd vs. Logstash: A Comparison of Log Collectors
- The Top 5 Elasticsearch Mistakes & How to Avoid Them

However, this ease of deployment and use can become a problem if Elasticsearch is not managed well. The JSON-based configuration is easy but if you want to specify comments for each and every configuration inside the file, then it is not for you.

The latest version of Solr provides a good set of Rest APIs that remove the complexities in the previous versions such as when creating custom sharded collections via a collections API, documenting clustering algorithms, and doing custom sharding. Overall, if your app is using JSON, then Elasticsearch is a better option. Otherwise, use Solr since its schema.xml and solrconfig.xml are very well documented.

Indexing and Searching

Data Sources

Solr accept data from different sources including XML files, comma-separated value (CSV) files, and data extracted from tables in a database as well as common file formats such as Microsoft Word and PDF. Elasticsearch also accepts data from many different sources such as ActiveMQ, AWS SQS, DynamoDB (Amazon NoSQL), FileSystem, Git, JDBC, JMS, Kafka, LDAP, MongoDB, neo4j, RabbitMQ, Redis, Solr, and Twitter. There are various plugins available as well.

Searching

Solr is much more oriented towards text search while Elasticsearch is often used for analytical querying, filtering, and grouping. The team behind Elasticsearch is always trying to make these queries more efficient (through methods including the lowering of memory footprint and CPU usage) and improve performance at both the Lucene and Elasticsearch levels. When comparing both, it's clear that Elasticsearch is a better choice for applications that require not only text search but also complex time series search and aggregations.

Both search engines use various analyzers and tokenizers that break up text into terms or tokens that are then indexed. Elasticsearch allows you to specify the query analyzer chain, which is comprised of a sequence of analyzers or

tokenizers on a per-document or per-query basis. This helps when you have multiple analyzers attached so that the output of one analyzer becomes the input of a second analyzer. In contrast, Solr does not support this feature.

Indexing

You can index both search engines while simultaneously using stopwords and synonyms to match documents. In Solr, the join index has to be a single-shard and replicated across all nodes to search inter-document relationships (such as SQL joins, for example). In the case of Elasticsearch, you can retrieve such related documents using **has_children** and **top_children** queries that make it more efficient. This helps to find the parent documents that have child documents that match the criteria. According to some performance tests, Elasticsearch may tend to produce better results than Solr in terms of indexing.

Scalable and Distributed

Search engines have to deal with large systems with millions of documents. For that matter, the search engines should be replicable, modular, and scalable enough to allow clustering and distributed architecture.

Designed for the Cloud

Elasticsearch is simple to scale and attracts use cases where large clusters are required. Solr — in its Elasticsearch-like fully distributed SolrCloud deployment mode — depends on Apache ZooKeeper. Although ZooKeeper is mature and widely used, it's ultimately an entirely separate application. SolrCloud is designed to provide a highly available, fault-tolerant environment for distributing indexed content and query requests across multiple servers. With SolrCloud, data is organized into multiple pieces — shards — that can be hosted on multiple machines. The replicas will help to achieve redundancy as well as scalability and fault-tolerance.

In comparison, Elasticsearch has a built-in, ZooKeeper-like component called Zen that uses its own internal coordination mechanism to handle the cluster state. ZooKeeper is better at preventing inconsistent states from arising due to the split-brain problem in Elasticsearch clusters. Since Elasticsearch is easy to start in a cluster and designed for the cloud, it would be the preferred choice as long as the inconsistent state issue is handled well.

Shard Splitting and Rebalancing

Shards are the partitioning unit for the Lucene index, and both Solr and ElasticSearch use them. You can distribute your index by running shards on different machines in a cluster. Until a couple of years ago, neither database allowed you to change the number of shards in your index — so if you wanted to add new shards to your existing setup, it was not permitted and you had to do a completely new setup. With the introduction of SolrCloud, Solr started supporting shard splitting, which allows you to add more shards by splitting existing shards. In comparison, ElasticSearch still does not support this and, in fact, actually discourages the practice.

If you have done proper capacity planning, you will know your future growth and the resulting needs for your Elasticsearch machines. By adding more machines to your setup, you can use the automatic shard-balancing feature within Elasticsearch. This will also help solve the shard-splitting issue.

To prepare your current machine for future sharding and the addition of more machines, you should have multiple shards in the current machines by splitting your index based on the estimated number of future machines required. The advantage is that each machine will have multiple shards, and when you add new machines, ElasticSearch will automatically balance the load and move shards to new nodes in the cluster. This automatic shard-rebalancing behavior is not available in Solr.

In comparison, Solr allows shards to be added (when using implicit routing) or split (when using composite ID), but shards cannot be removed. It does allow you to increase the replicas.

In Elasticsearch, each index has five shards by default. It does not allow you to change the number of primary shards, but it does allow you to increase the number of replicas. Automatic shard rebalancing is useful for horizontal scaling. When a new machine is added, it will automatically rebalance the shards that are available in different machines.

The Community

Solr has a broad, open-source community. Anyone can contribute to Solr, and new Solr developers or code committers are elected based on merit only. Elasticsearch is technically open-source but not fully. All contributors have access to the source code, and users can make changes and contribute them. But final changes are confirmed and done by employees of Elastic (the company that runs Elasticsearch and other software). Therefore, Elasticsearch is driven more by a single company rather than a whole community.

Solr contributors and committers span multiple organizations while Elasticsearch committers are from Elastic only. It's also been observed that Solr's strong community has a healthy project pipeline and many well-known companies that take part. These members also invest in the platform by contributing throughout the entire development and engineering process.

Both have great user bases as well as rich developer communities, but ElasticSearch is newer in comparison to Solr. Solr has been around for a much longer period of time, so its ecosystem is well-developed and has a larger user base.

The Documentation

Solr scores big here. It is a very well-documented product with clear examples and contexts for API use cases. Elasticsearch's documentation is organized, but it lacks good examples and clear configuration instructions.

For Elasticsearch, some examples are written in YAML and some are in JSON. A number of discrepancies between the code and what is documented on the website have also been observed.

In comparison, Solr is consistent and very well-documented. Without going deep into code, you can learn much more about indices, sharding, and searching.

So, Solr or Elasticsearch?

Sometimes it's tough to identify a clear winner. Whether you select Solr or Elasticsearch, you first need to understand your proper use case and future needs. To summarize each of their attributes:

	Solr	**Elasticsearch**
Installation and Configuration	Supported by detailed documentation	More intuitive
Indexing/Searching	Text-oriented	Better performance of analytical queries
Scalability and Clustering	Provides SolrCloud	Better inherent scalability and designed for the cloud
Community	A much bigger ecosystem of community	A growing community though not a complete open source mindset
Documentation	Very well-documented	Lacks in documentation

Remember:

- Elasticsearch is more popular among newer developers due to its ease of use. But if you are already used to working with Solr, stay with it because there is no specific advantage of migrating to Elasticsearch
- If you need it to handle analytical queries in addition to searching text, Elasticsearch is the better choice
- If you need distributed indexing, then you need to choose Elasticsearch. Elasticsearch is the better option for cloud and distributed environments that need good scalability and performance

In summary, both are feature-rich search engines and more or less give the same performance as long as they are designed and implemented well.

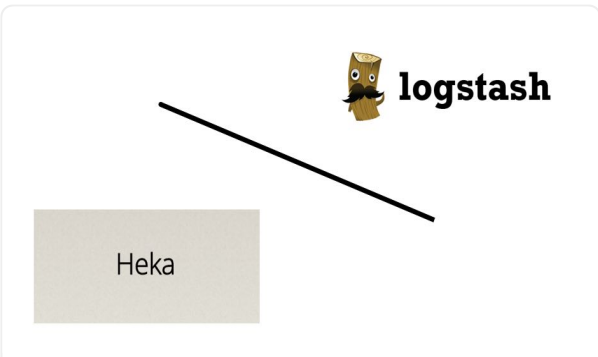
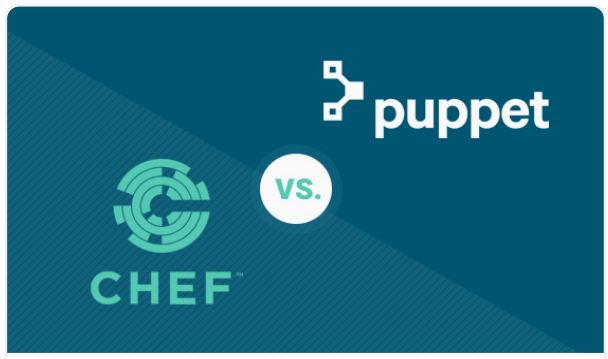
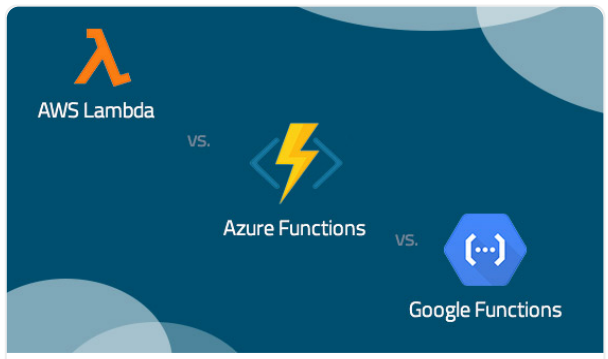
Need Help Managing Elasticsearch? Logz.io Can Do the Heavy Lifting for you.

Try Logz.io Today!

5 SHARES

DevOps News and Tips Straight to your Inbox

Email Subscribe

By submitting this form you are accepting our Privacy Policy

Related Posts